

Weston Ruter
4:37 am on November 18, 2025

WordPress 6.9 Frontend Performance Field Guide

This post is the latest in a series of updates focused on the performance improvements of major releases (see [6.8](#), [6.7](#), [6.6](#), [6.5](#), [6.4](#), [6.3](#), and [6.2](#)).

WordPress 6.9 is the second and final major release of 2025. It includes numerous improvements to the performance of loading pages on the frontend:

- Scripts: improve script loading performance by adding support for fetchpriority, printing script modules in the footer, and optimizing the emoji detection script.
- Styles: optimize loading of stylesheets by loading block styles on demand in classic themes, omitting styles for hidden blocks, increasing the inline style limit from 20K to 40K, and inlining minified stylesheets in block themes.
- Introduce the template enhancement output buffer to implement optimizations previously impossible (such as the aforementioned on-demand style loading in classic themes).
- More: spawn WP Cron at shutdown, eliminate layout shifts in the Video block, fix RSS feed caching, and so on.

The performance changes in this release include [38 Trac tickets](#) (26 enhancements, 11 defects, 1 task) and [31 Gutenberg PRs](#), although this post does not describe improvements to the performance of the editor nor the database query and caching optimizations. This post highlights the key changes to the frontend for site visitors, as well as looking at their impact in terms of web vitals metrics, such as TTFB, FCP, and LCP.

1. Performance Improvements

1. Support specifying fetchpriority for scripts and script modules
2. Support printing script modules in the footer
3. Convert emoji detection script to script module and move to footer
4. Increase inline style limit from 20K to 40K bytes
5. Minify and inline block theme stylesheets
6. Omit styles (and scripts) for hidden blocks by default
7. Load block styles on demand in classic themes
8. Introduce the template enhancement output buffer
9. Spawn WP Cron at shutdown instead of init
10. Improve RSS feed caching
11. Eliminate Layout Shift in Video Block
12. Summary of Performance Improvements

2. Performance Metrics

1. Web Vitals
 1. “Sample Page”
 2. “Block category: Common”
2. Server Timing
 1. Classic Theme (Twenty Twenty-One)
 2. Block Theme (Twenty Twenty-Five)

3. What’s next?

Performance Improvements

Support specifying fetchpriority for scripts and script modules

Trac: [#61734](#), [#64194](#)

Problem: Browsers load scripts with high priority which adds contention for the critical rendering path.

As of 6.3, WordPress started adding the `fetchpriority=high` attribute to `IMG` tags which it determines are likely to be the Largest Contentful Paint (LCP) element (see [#58235](#)). This `fetchpriority` attribute is also supported on elements beyond `IMG`, and it can have other values besides `high`, namely `low`, and `auto`. As a way to complement adding `fetchpriority=high` to the likely LCP `IMG` element, a way to further prioritize loading of that LCP image resource is to add `fetchpriority=low` to deprioritize other elements which are known to not be in the critical rendering path. In 6.8 this ability to specify `fetchpriority` is added for scripts and script modules (and `modulepreload` links).

By default, browsers load scripts with a high priority, and this is especially true for blocking scripts. Browsers diverge in their default fetch priorities for classic scripts with `defer` or `async` loading strategies, but they all load script modules with a high priority even though they intrinsically have a deferred loading. Blocks have had their view scripts printed in the `<head>`, [reasoning](#) that it allowed them to be discovered earlier. However, now that all [core](#) blocks use [the Interactivity API which requires script modules](#), this means that they are adding [network](#) contention for loading any LCP element resource. This is especially wasteful for blocks using the Interactivity [API](#) since they use [server-side rendering](#) which by nature moves their scripts out of the critical rendering path.

So in WordPress 6.9, script modules for interactive blocks (using the Interactivity API) get `fetchpriority=low` by default. This includes custom blocks not in core. In addition to `fetchpriority=low` being the default for interactive blocks' view script modules, it is also the default for the `comment-reply` classic script.

In terms of the API for `WP_Scripts`, a `fetchpriority` key is added to the `$args` array passed to `wp_register_script()` and `wp_enqueue_script()`. The valid values are `auto`, `low`, and `high`. The default value is `auto`, which browsers usually treat the same as `high` (unless the script also has `defer` or `async`). The default is subject to change in future versions. To register and enqueue a script to be printed in the footer with a low fetch priority:

```
wp_enqueue_script(
    'foo',
    plugins_url( 'foo.js', __FILE__ ),
    array(),
    '1.0',
    array(
        'in_footer' => true,
        'strategy'  => 'defer',
        'fetchpriority' => 'low',
    )
);
```

This results in the following being printed:

```
<script
  src="https://example.org/wp-content/plugins/example/foo.js?ver=1.0"
  id="foo-js"
  defer
  data-wp-strategy="defer"
  fetchpriority="low"
></script>
```

You may change the fetch priority for an existing script as follows:

```
wp_script_add_data( $handle, 'fetchpriority', 'low' );
```

In the same way that the introduction of script loading strategies in [#12009](#) introduced `$args` as the 5th parameter to `wp_register_script()` and `wp_enqueue_script()`, the same has been done for script modules in the `wp_register_script_module()` and `wp_enqueue_script_module()` functions (as well as `WP_Script_Modules::register()` and `WP_Script_Modules::enqueue()`). The valid values are the same as with scripts above. To register a script module with a fetch priority of low:

```
wp_register_script_module(
    'foo',
    plugins_url( 'foo.js', __FILE__ ),
    array(),
    '1.0',
    array(
        'fetchpriority' => 'low',
    )
);
```

To change the priority of an existing script module, you can use the new `WP_Script_Modules::set_fetchpriority()` method:

```
wp_script_modules()->set_fetchpriority( $handle, 'low' );
```

The `fetchpriority` for an enqueued script or script module propagates back to any dependencies, overriding any assigned `fetchpriority` on the dependency. This includes the `modulepreload` links for script modules. For example, the following script module dependency `bar` lacks any explicit `fetchpriority` (and thus defaults to `auto`), but there is a dependent script module `foo` which has an explicit fetch priority of low:

```
wp_register_script_module(
    'foo',
    plugins_url( 'foo.js', __FILE__ ),
    array(),
    '1.0',
    array(
        'fetchpriority' => 'low',
    )
);
```

To change the priority of an existing script module, you can use the new `WP_Script_Modules::set_fetchpriority()` method:

```
wp_script_modules()->set_fetchpriority( $handle, 'low' );
```

The `fetchpriority` for an enqueued script or script module propagates back to any dependencies, overriding any assigned `fetchpriority` on the dependency. This includes the `modulepreload` links for script modules. For example, the following script module dependency `bar` lacks any explicit `fetchpriority` (and thus defaults to `auto`), but there is a dependent script module `foo` which has an explicit fetch priority of low:

```
wp_register_script_module( 'bar', plugins_url( 'bar.js', __FILE__ ), '1.0' );
wp_enqueue_script_module(
    'foo',
    plugins_url( 'foo.js', __FILE__ ),
    array( 'bar' ),
    '1.0',
    array(
        'fetchpriority' => 'low',
    )
);
```

The resulting markup on the page is as follows, showing both get `fetchpriority=low`:

```
<script type="importmap" id="wp-importmap">
  {
    "imports": {
      "bar": "https://example.org/wp-content/plugins/example/bar.js?ver=1.0"
    }
  }
</script>
<script
  type="module"
  src="https://example.org/wp-content/plugins/example/foo.js?ver=1.0"
  id="foo-js-module"
  fetchpriority="low"
></script>
<link
  rel="modulepreload"
  href="https://example.org/wp-content/plugins/example/bar.js?ver=1.0"
  id="bar-js-modulepreload"
  fetchpriority="low"
>
```

For additional background and performance metrics, see [Improve LCP by Deprioritizing Script Modules from the Interactivity API](#) (personal [blog](#)).

Impact: When testing a site with the Twenty Twenty-Five theme on a page with one interactive block (the Navigation block) and a [featured image](#) (the LCP element), the addition of `fetchpriority` was seen to improve LCP by a median ~8% over an emulated broadband connection.

Support printing script modules in the footer

Trac: [#63486](#)

Problem: Non-essential scripts in the `head` add network contention for resources in the critical rendering path.

Related to the preceding enhancement where `fetchpriority` support was added to scripts, another deprioritization enhancement was added to `WP_Script_Modules` in this release: the ability to print script modules in the footer. This improvement brings `WP_Script_Modules` in closer alignment with `WP_Scripts`. The `in_footer` key in the `$args` parameter for `wp_register_script()` and `wp_enqueue_script()` is now also supported on `wp_register_script_module()` and `wp_enqueue_script_module()`. As with `WP_Scripts`, the default value for `in_footer` remains `false`. For example:

```
wp_enqueue_script_module(
    'foo',
    plugins_url( 'foo.js', __FILE__ ),
    array(),
    '1.0',
    array(
        'in_footer' => true,
    )
);
```

Results in the following being printed at `wp_footer`:

```
<script
  type="module"
  src="https://example.org/wp-content/plugins/example/foo.js?ver=1.0"
  id="foo-js-module"
></script>
```

The `modulepreload` links for any script module static dependencies will remain printed at `wp_head`.

The location at which an existing script module is printed can be updated via the new `WP_Script_Modules::set_in_footer()` method, for example:

```
wp_script_modules()->set_in_footer( $handle, true );
```

Being able to print script modules in the footer is currently only relevant to block themes, as classic themes already printed all script modules in the footer. However, as seen below with [loading block styles on demand in classic themes](#), the [template enhancement output buffer](#) could be used to hoist script modules originally intended for the `<head>`.

As referenced in the previous section, for additional background and performance metrics, see [Improve LCP by Deprioritizing Script Modules from the Interactivity API](#) (personal blog).

Impact: Since a script module is loaded with a `high` fetch priority by default, simply moving it to the footer reduces network contention for resources in the critical rendering path (e.g. the LCP image). Similarly to the above benchmarks with adding `fetchpriority=low`, printing a script module in the footer improves LCP by a median ~7% when benchmarking a page in Twenty Twenty-Five with the Navigation block and the LCP element being the featured image, over an emulated broadband connection. When a script module is registered with *both* `fetchpriority=low` *and* `in_footer=true` then the LCP reduction was measured to be ~9% with the same conditions, so a ~2% improvement when combined.

Convert emoji detection script to script module and move to footer

Trac: [#64076](#), [#63842](#)

Problem: The emoji detection script was adding 3KB of inline blocking [JS](#) to the critical rendering path.

As a follow-up to [#58472](#) from WordPress 6.3, the emoji detection script (i.e. emoji loader) has been further optimized to be removed from the critical rendering path. While the detection script has always been an inline script which minimized its performance impact, a non-module inline script still blocks the [HTML](#) parser from continuing until the script has executed. This contributes to a render delay and thus a degraded LCP. The emoji detection script serves as a fallback in case the operating system doesn't support the latest emoji, and even then any fallback emoji image it loads would not be the LCP element. Lastly, the detection script waits until `DOMContentLoaded` anyway to initiate any emoji loading. All of this contributes to the need to deprioritize the emoji detection script.

In WordPress 6.9, the emoji detection script has been converted from a classic (blocking) inline script to an inline script module. This prevents the script from blocking the HTML parser. Furthermore, since a script module is deferred to be executed when the DOM has been fully constructed, it has no benefit being located in the `head`, and so it is moved to the footer. It is much better to reserve that “head -room” for code which is part of the critical rendering path, and to reallocate that ~3KB from the emoji detection script to instead inline additional [CSS](#) (see [below](#)). In the future it could be converted from an inline script module to an external one, per [#64259](#).

As part of the emoji detection script modernization, the emoji settings are no longer injected into the inline script but are instead exported as [JSON](#) into an `application/json` script. This follows the recent “pull” pattern for exporting data for script modules via the [script module data {\\$module_id}](#) filter. There is a slight chance for a back-compat breakage here for any plugins that may depend on the `_wpemojiSettings` global variable. The emoji detection script parses the JSON out of `script#wp-emoji-settings` when it executes and immediately populates the `_wpemojiSettings` global variable so it will be defined by the time the `DOMContentLoaded` event fires. But if a [plugin](#) expects this variable to be defined *before* `DOMContentLoaded` (which is unlikely), then to remain compatible it should be updated to obtain the emoji settings as follows:

```
let wpEmojiSettings = window._wpemojiSettings;
if ( ! wpEmojiSettings ) {
    const settingsScript = document.getElementById( 'wp-emoji-settings' );
    if ( settingsScript ) {
        wpEmojiSettings = JSON.parse( settingsScript.text );
    }
}
```

Some sites remove the emoji detection script altogether to improve performance. For example, this commonly involves the following [PHP](#) code:

```
remove_action( 'wp_head', 'print_emoji_detection_script', 7 );
```

For backwards compatibility, this remains a valid way to prevent this script from being printed. Unhooking that `print_emoji_detection_script()` function from running prevents it from adding an action to print the emoji detection script at `wp_print_footer_scripts`.

Impact: The performance impact of converting the emoji detection script to a script module and moving it from `wp_head` to `wp_footer` is largely imperceptible on a high-end or mid-range device. But on a low-tier mobile phone, the impact can be measured. Over 100 page loads to the Sample Page in the Twenty Twenty-Five theme while emulating a low-tier mobile phone, switching to a script module improves FCP (First Contentful Paint) by ~7% and LCP by ~4%. When also moving the emoji detection script to the footer, an additional 1% to 2% improvement to FCP and LCP are measured when emulating a Fast 4G network connection. Note that the long task identified previously in [#58472](#) was also noticed while profiling on a lower-powered device.

Increase inline style limit from 20K to 40K bytes

Trac: [#63018](#), [#64178](#)

Problem: External stylesheets are render-blocking which negatively impacts FCP and LCP.

There were a couple key [block-styles loading enhancements](#) introduced back in WordPress 5.8:

- 1. [Inlining small assets](#)
- 2. [Only load styles for used blocks](#)

The first involved splitting up the large `wp-block-library` stylesheet (120KB minified) into separate stylesheets for each block. The overall amount of CSS added to the page could then be greatly reduced by just loading the styles on demand for the blocks actually used. (This ability to load separate block styles on demand was limited to block themes, but see [below](#) where this is enabled for classic themes in 6.9 as well.) Given that the separate block stylesheets are often quite small (1.5KB average and 300B median), it makes sense to inline them to avoid the negative performance impact of external stylesheets which are [render-blocking](#).

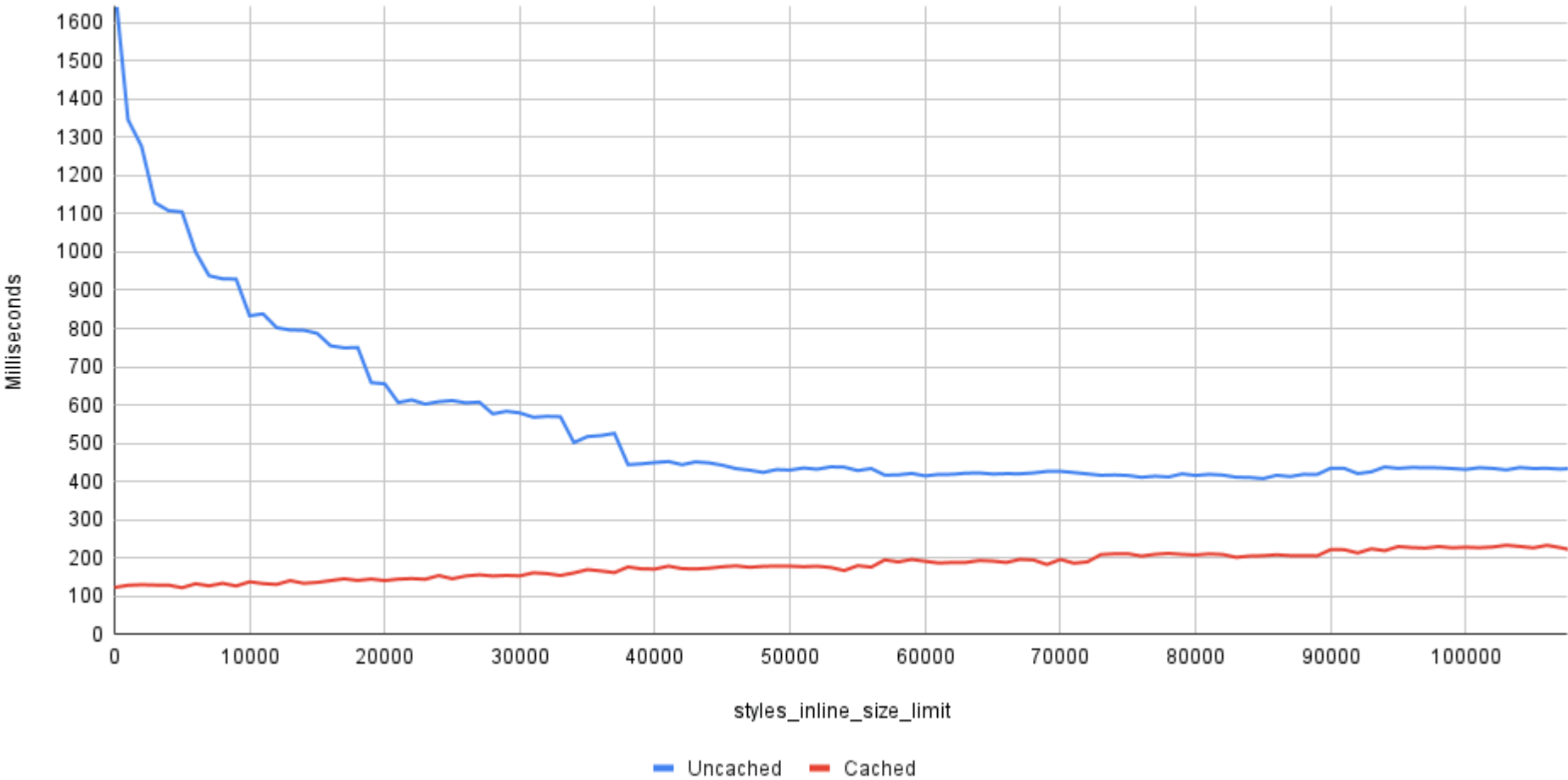
► Minified block stylesheet file sizes

To inline small stylesheets, WordPress iterates over all enqueued styles which have a `path` data supplied for where the stylesheet can be found on the filesystem. It then reads each CSS file and converts it into an inline style, up until the total amount of inlined CSS reaches the default limit of 20KB. (There is no caching currently involved in `wp_maybe_inline_styles()`.) Inlining of styles is not limited to block stylesheets. Any stylesheet with `path` data added is able to be inlined. This limit can be customized via the `styles_inline_size_limit` filter.

The 20KB default limit for inline CSS was a conservative starting point which was intended to be raised in the future. While inlining 100% of CSS can maximize the page load time improvement for first-time visitors, a key concern is how inlining can negatively impact repeat visitors. When all CSS is loaded from external stylesheets, any repeat page views result in that CSS being loaded from the browser cache. When all the CSS is inlined, however, first time visitors and repeat visitors alike have to re-download all the CSS for every page load. What is needed is to measure the LCP impact of increasing the limit of inline CSS on both initial and repeat page loads. This was [done in #63018](#). When benchmarking with network conditions emulating either broadband or Fast 4G, the results are similar. Page load time grows linearly for cached (repeat) visits whereas page load time decreases with exponential decay for initial (uncached) visits. The point at which the exponential decay reaches the point of diminishing returns is when the `styles_inline_size_limit` is around 40KB:

Median LCP-TTFB for Uncached vs Cached page loads with increasing `styles_inline_size_limit`

Emulating Fast 4G mobile connection on a Moto G4 device. Request count: 10. All external stylesheets are inlined once limit is above 109KB.



Impact: On a Fast 4G connection, increasing the limit from 20 to 40 KB:

- Cached visits: LCP-TTFB increases from 141.3 to 171.6 milliseconds (+30.3 ms, +21.44%)
- Uncached visits: LCP-TTFB decreases from 655.7 to 449.9 milliseconds (-205.8 ms, -31.39%)

So the percentage improvement for initial visits is larger than the percentage [regression](#) for repeat visits, and the relative improvement for initial visits (-205.8 ms) is much larger than the relative regression for repeat visits (+30.3 ms).

► A note regarding oEmbed discovery links

Minify and inline block theme stylesheets

Trac: [#63012](#), [#63007](#)

Problem: Stylesheets used in two core block themes were unminified and unable to be inlined, causing render-blocking that negatively impacted FCP and LCP.

Block themes generally do not need to define their own stylesheets, since the styles come from [global styles](#) (`theme.json`) and the existing separate block styles. Indeed, the bundled themes Twenty Twenty-Three and Twenty Twenty-Four do not enqueue any CSS file. With the increase in the inline

CSS limit (see [above](#)), this means such themes can load without any render-blocking resources, which greatly improves LCP. Nevertheless, the Twenty Twenty-Two and Twenty Twenty-Five themes do enqueue their own stylesheets, although they are quite small when minified: ~2.5 KB and ~600 B respectively. The stylesheets' small size make them prime candidates for inlining, but until now they haven't for two reasons: no stylesheets for bundled themes have been minified and none have the `path` data supplied.

In [#63012](#), CSS minification has been introduced specifically for the core block themes, and the `path` data was added in [#63007](#). In the same way as core, the minified styles (`style.min.css`) are served by default, with the unminified versions (`style.css`) served when `SCRIPT_DEBUG` is enabled.

As a reminder, site owners are highly discouraged from using the Theme File Editor in the [admin](#) to modify files in themes installed from the directory (an additional warning was added in [#63012](#) for this). As of the latest versions of Twenty Twenty-Two and Twenty Twenty-Five, changes to the `style.css` will not appear for site visitors by default since they will be served `style.min.css` instead. Site owners are strongly encouraged to use the Site Editor to supply additional styles instead. (A new admin notice about this appears in the Theme File Editor when modifying a CSS file.) If these core themes are forked, changes to the `style.css` can be re-minified using the newly-distributed `package.json` via `npm install && npm run build`.

Minifying the CSS in bundled classic themes is being tracked in [#64109](#).

Impact: When benchmarking the Sample Page in a stock WordPress install, eliminating all render-blocking stylesheets from Twenty Twenty-Five can improve LCP by ~36% on an emulated Fast 4G connection. And on an emulated Slow 3G connection, inlining all styles can improve LCP by ~45%! For more information, see [Minified CSS Inlining](#) (personal blog).

Omit styles (and scripts) for hidden blocks by default

Trac: [#63676](#)

Problem: Scripts and styles for hidden blocks were being added to the page even though unused.

In WordPress 5.8 (via [#50328](#)), core added the ability to conditionally print scripts and styles for blocks which are actually used on the page. (Initially this only applied to block themes, but as of [#64099](#) it also applies to classic themes, per [below](#).) Nevertheless, when a block didn't output any markup, its assets would still get printed. This could be easily seen on block themes when a featured image was not assigned to a post but the template includes a Featured Image block. When looking at the single template for that post, you'd see the stylesheet for the Featured Image block in the HTML page, even though the block is absent:

```
<style id='wp-block-post-featured-image-inline-css'>
.wp-block-post-featured-image{
/* ... */
}</style>
```

The same would be true for the Site Logo and Tagline blocks when they appear in the template (as they are in Twenty Twenty-Five) but have no logo or tagline set. Likewise, when a post has comments disabled, the CSS for the Comments block would still appear in the page even when no comments are present. These stylesheets amount to ~5KB of unused CSS. This issue becomes more pronounced in 6.9 with the introduction of [Block Visibility support](#) via [#64061](#). (The issue could be observed with the Block Visibility plugin as well.) When a block is hidden in the editor, its scripts and styles would still get needlessly enqueued. This would negatively impact the performance of the page.

So in 6.9, after a block is rendered, any scripts and styles that it enqueues will get dequeued if it turns out that the block is not ultimately printed. This works with nested blocks, so if you have a Video block nested deep inside a Group block, and yet you hide the Group block, then the styles for the Video block will be omitted from the page.

In some cases, a site might depend on the assets (scripts and styles) associated with a hidden block. To accommodate this unusual scenario, a new `enqueue_empty_block_content_assets` filter is introduced to force scripts and styles for hidden blocks to be enqueued. For example, to force the assets for all hidden blocks to be enqueued:

```
add_filter( 'enqueue_empty_block_content_assets', '__return_true' );
```

The block name is supplied as the second argument to this filter, so your filter callback can conditionally force the assets for a specific hidden block to be enqueued. For example, to enqueue the assets for the Featured Image block even when there is no featured image assigned:

```
add_filter(
    'enqueue_empty_block_content_assets',
    function ( $enqueue, $block_name ) {
        if ( 'core/post-featured-image' === $block_name ) {
            $enqueue = true;
        }
        return $enqueue;
    },
    10,
    2
);
```

This should only be done as needed.

Impact: This can reduce 5KB or more of CSS on the page, making room for inlining critical CSS. This also reduced unused JavaScript, potentially improving TBT (Total Blocking Time) and INP (Interaction to Next Paint) metrics.

Load block styles on demand in classic themes

Trac: [#64099](#), [#64150](#)

Problem: Classic themes add much more CSS than may be actually needed on a page.

As mentioned [above](#), one of the key [block-styles loading enhancements](#) introduced back in WordPress 5.8 was to [only load styles for used blocks](#). However, this enhancement was limited to block themes due to the difference in how block themes and classic themes construct their templates. In block themes, all of a template’s blocks are [rendered *before*](#) any HTML is constructed in `template-canvas.php`. This means that a block theme can load block assets on demand since the needed styles are discovered before the `wp_head` action has fired. Templates in classic themes work differently: They procedurally generate the HTML, printing the `<head>` before rendering anything that appears in the `<body>`. This has meant that classic themes have needed to load the aforementioned large 120KB `wp-block-library` stylesheet up front because they don’t know which blocks will appear in the page. Classic themes could actually still opt-in to loading separate block styles on demand via:

```
add_filter( 'should_load_separate_core_block_assets', '__return_true' );
```

But this had a large downside: The separate block styles would then get loaded in the footer, meaning there would be a flash of unstyled content (FOUC).

In WordPress 6.9, classic themes now opt-in to loading separate block styles on demand by default. This is achieved via the template enhancement output buffer (see [below](#)). While the template output is being output-buffered, a classic theme prints the minimal common styles at `wp_head` and then proceeds with rendering the rest of the template. When it gets to printing the late-enqueued styles (i.e. from rendered blocks), it captures the output when being printed at `wp_footer` and then hoists the late-printed `<link>` and `<style>` tags up to the `HEAD`. Styles related to blocks are inserted right after the `wp-block-library` common stylesheet (to preserve the original CSS cascade), and any other late-enqueued styles are inserted at the end of the `<head>`. This insertion is done using the HTML Tag Processor.

In addition to reducing the amount of CSS added for blocks, **plugin authors** are encouraged to start enqueueing styles at the point where they are used on the page, knowing that core will move the stylesheet to the `<head>`.

Impact: The difference in the amount of CSS printed to a page can be substantial. For example, see the kilobytes of CSS added to the Sample Page in each of the classic core themes in 6.8 vs 6.9:

Theme	WP 6.8	WP 6.9	Diff (KB)	Diff (%)
twentyten	159	54	-105	-66%
twentyeleven	195	80	-114	-59%
twentytwelve	193	85	-108	-56%
twentythirteen	258	152	-106	-41%
twentyfourteen	258	152	-106	-41%
twentyfifteen	299	193	-106	-35%
twentysixteen	258	151	-107	-41%
twentyseventeen	236	122	-114	-48%
twentynineteen	361	256	-105	-29%
twentytwenty	257	153	-104	-40%
twentytwentyone	289	184	-105	-36%

There is an average of 45% less CSS on the Sample Page across bundled themes in 6.9.

When evaluating a URL that has many more blocks on it, for example the “Block category: Common” post from the theme unit test data, the amount of savings is reduced, as expected:

Theme	WP 6.8	WP 6.9	Diff (KB)	Diff (%)
twentyten	160	99	-61	-38%
twentyeleven	195	127	-68	-35%
twentytwelve	193	130	-63	-33%
twentythirteen	258	197	-61	-24%
twentyfourteen	258	197	-61	-24%
twentyfifteen	299	238	-61	-20%
twentysixteen	259	196	-62	-24%
twentyseventeen	236	175	-61	-26%
twentynineteen	362	301	-61	-17%
twentytwenty	257	197	-60	-23%
twentytwentyone	290	229	-61	-21%

This is still an average reduction of 26% less CSS.

Not only is the amount of CSS reduced, but as mentioned above, up to 40KB of this CSS can now be inlined which further reduces render-blocking on a page.

There is a cost to using an output buffer to reduce the amount of CSS: There is an increase to the TTFB. As referred to above, classic themes have historically generated their templates procedurally; this has meant they can start streaming output earlier. So even though it is expected for the TTFB to be worse in 6.9 for classic themes, this is done for the sake of an overall improvement to LCP (for which TTFB is the first component). The increase in TTFB for classic themes brings them closer to the TTFB for block themes. For classic themes, the average improvement to the LCP for the Sample Page is ~4% (with LCP-TTFB improved by ~6%).

If the increase in TTFB is not acceptable for your site, you may opt out via the `should_load_separate_core_block_assets` filter:

```
add_action(
    'after_setup_theme',
    function () {
        // Only do this if absolutely necessary!
        add_filter( 'should_load_separate_core_block_assets', '__return_false' );
    }
);
```

This must be done before the `init` action, at which point the new `wp_load_classic_theme_block_styles_on_demand()` function will otherwise opt in a classic theme to load separate block styles by default.

Otherwise, if you notice that you are now missing some expected styles, it could be that you have content which is not being run through `do_blocks()` and thus it is not being enqueued on demand. Beyond ensuring your block markup is being run through `do_blocks()` (which is a known issue for the Latest Posts block), you can fix this by either explicitly enqueueing the necessary separate block-specific styles. For example, to enqueue the stylesheet for the Video block:

```
add_action(
    'wp_enqueue_scripts',
    function () {
        wp_enqueue_style( 'wp-block-video' );
    }
);
```


As a last resort, you can opt out of loading separate assets on demand using the `should_load_separate_core_block_assets` filter shown above. This means you'll go back to getting the large `wp-block-library` stylesheet, and you won't see the expected LCP improvement.

- As a workaround for sites experiencing any issue with loading separate core block assets, there is a small single-file plugin that allows you to remain on 6.9 but opt out of loading separate core block styles: **Load Combined Core Block Assets**. This plugin includes the ability for you to test loading separate core block styles via a query parameter. Once 6.9.1 is released with the fix, the plugin could inform you to re-test, so that the plugin can be deactivated and uninstalled at that time.

If you added the following to your site via a theme or plugin:

```
add_filter( 'should_load_separate_core_block_assets', '__return_false' );
```

Please remove this, and instead install the plugin. This will help ensure you don't get stuck with loading the combined core block assets indefinitely. The use of combined core block assets means much more CSS is added to the page than is likely needed, and this can negatively impact page load time (e.g. FCP and LCP). By WordPress 6.9 loading separate block assets on demand, this also allows for small stylesheets to be inlined to further reduce render-blocking resources.

► A note regarding themes which try to opt out of all block styles

Introduce the template enhancement output buffer

Trac: [#43258](#), [#64108](#), [#64126](#)

Problem: Core did not facilitate making optimizations on the entire rendered template.

As introduced in the previous section, WordPress 6.9 for the first time includes an optional output buffer for rendered templates. Buffering the response output is extremely common in the plugin ecosystem, even in feature plugins maintained by WordPress core contributors:

- Gutenberg's Full Page Navigation experiment uses output buffering to add Interactivity API directives to the `<body>` tag.
- Performance Lab uses output buffering to send `Server-Timing` headers for timing code during template generation.
- Optimization Detective (part of the Performance Lab featured plugin suite) leverages output buffering so that extensions like Image Prioritizer can add `fetchpriority=high` on an actual LCP `IMG` element.

These plugins do output buffering of the overall HTML response so that they can manipulate the document (e.g. to apply optimizations) or so they can send headers after the document has been fully constructed, but before it has been sent. They all implement output buffering in different ways, however, starting at various WordPress actions or even filters, and each reinventing the wheel. Core introduces the **template enhancement output buffer** as a standardized way for extensions to do output buffering.

As seen with loading separate block styles on demand in classic themes, there is a tradeoff to using an output buffer. While the overall response time (i.e. Time to Last Byte, or TTLB) may be relatively unchanged, the TTFB may be significantly increased since WordPress is not able to stream the responses, for example sending the `<head>`, flushing, sending the page header, flushing, and so on until the document has been sent in its entirety. Streaming responses can provide a great user experience assuming that the template has been constructed in an optimal way. This is not the case for classic themes, however, as they have had to send an excessive amount of CSS up front since they do not know what the page will need. So for classic themes, the benefits of the output buffer can outweigh the increased TTFB if it means separate block styles can be loaded on demand. Again, see the previous section. Furthermore, enabling the output buffering for block themes doesn't significantly increase TTFB because block templates are already essentially output buffered since a page's blocks are rendered prior to starting to send the HTML response. In any case, enabling the template enhancement output buffer may not affect your TTFB in practice due to page caching being employed on a site. Caching plugins also use output buffering.

The template enhancement output buffer is started immediately before the template is loaded. After the `template_include` filter has applied and before `include $template` runs, a new `wp_before_include_template` action fires. It is during this action that the output buffer starts. Nevertheless, the output buffer is only started if there is code that will use it. There are two key hooks introduced for making use of output buffering:

- The `wp_template_enhancement_output_buffer` filter is passed the entire output buffer HTML, as well as the initial output buffer before the filters have applied. Extensions can use this to manipulate the response (ideally with the HTML API). This filter only applies when the response has an `HTML Content-Type` response header.
- The `wp_finalized_template_enhancement_output_buffer` action fires after the previous filters have applied. This action is passed the final HTML response prior to it being sent to the browser. This action is the last chance to send an HTTP response header.

The output buffer will only be started by default if there is a callback added to either this filter or this action. For example, the Optimization Detective plugin mentioned above will be updated to use this filter. You may also force the output buffer on or off using the new `wp_should_output_buffer_template_for_enhancement` filter with a callback that returns a boolean value. The `wp_should_output_buffer_template_for_enhancement()` function returns the value for this applied filter. Once the output buffer has been started, the `wp_template_enhancement_output_buffer_started` action fires.

It is called a template *enhancement* output buffer because it is intended that site owners be able to opt out of the buffer if they would rather stream responses. Treat it as a progressive enhancement. Therefore, the output buffer should be used for doing things like optimizing the page or adding analytics. It is not intended for adding or removing arbitrary content.

As an example of filtering the output buffer, the following code makes sure that the first five `IMG` tags in the response aren't lazy-loaded, whereas the remaining `IMG` tags are loaded lazily:

```
add_filter(
    'wp_template_enhancement_output_buffer',
    function ( $buffer ) {
        $processor = new WP_HTML_Tag_Processor( $buffer );
        $img_index = 0;
        while ( $processor->next_tag( 'IMG' ) ) {
            if ( $img_index < 5 ) {
                $processor->remove_attribute( 'loading' );
            } else {
                $processor->set_attribute( 'loading', 'lazy' );
            }
            $img_index++;
        }
        return $processor->get_updated_html();
    }
);
```

Important: Avoid using regular expressions to attempt to parse the output buffer. Regular expressions are notoriously brittle for HTML and there are countless edge cases to consider. Fortunately, WordPress's [HTML API](#) (`WP_HTML_Tag_Processor` and `WP_HTML_Processor`) has been designed as an efficient and robust way to process HTML. If you need to make manipulations beyond the current scope of the HTML API, consider doing so in the DOM via the [Dom\HTMLDocument](#) in PHP 8.4 which is a fully compliant HTML5 parser.

Here's an example of using the `wp_finalized_template_enhancement_output_buffer` action to implement [conditional requests](#) for template responses (which WordPress similarly implements for feeds):

```
add_action(
    'wp_finalized_template_enhancement_output_buffer',
    function ( $buffer ) {
        $etag = '' . md5( $buffer ) . '';
        header( "ETag: $etag" );
        header( 'Cache-Control: public, max-age=3600' );

        if (
            isset( $_SERVER['HTTP_IF_NONE_MATCH'] ) &&
            wp_unslash( $_SERVER['HTTP_IF_NONE_MATCH'] ) === $etag
        ) {
            status_header( 304 );
        }
    }
);
```

Important: You cannot print anything during this action. It will not be included in the response, and as of PHP 8.4, a deprecation warning will be issued:

```
PHP Deprecated: ob_end_flush(): Producing output from user output handler wp_finalize_template_enhancement_output_buffer is deprecated in /wp-content/themes/twentytwentyfour/inc/deprecated.php on line 10
```

If you want to include anything in the response, you must use the aforementioned `wp_template_enhancement_output_buffer` filter. If any callback for this filter or the `wp_finalized_template_enhancement_output_buffer` action causes a PHP deprecation, notice, warning, or error to be displayed due to `WP_DEBUG_DISPLAY` being enabled, then a custom error handler will capture the error and append the error to the end of the output buffer. Exceptions thrown during this filter or action will be caught and converted into warnings, with the unfiltered output buffer returned.

Also important: Callbacks for this filter or action must not attempt to call `ob_start()` to do any output buffering of their own. This will raise a fatal error:

```
PHP Fatal error: ob_start(): Cannot use output buffering in output buffering display handlers in /wp-content/themes/twentytwentyfour/inc/deprecated.php on line 10
```

Output buffering may continue to be used elsewhere during template rendering as usual. It has been tested with popular plugins that open output buffers earlier (e.g. caching plugins) and those which open buffers for template parts.

Spawn WP Cron at shutdown instead of init

Trac: [#63858](#)

During this release it was [discovered](#) that when WP Cron has been spawned at the `init` action, it may actually cause an additional second delay. It attempts to do an async non-blocking loopback request to `wp-cron.php`, but this `blocking => false` is [not working](#). Therefore, to prevent spawning WP Cron from increasing TTFB, this now happens at `shutdown` instead of `init`.

Impact: Possible reduction of TTFB by 1 second when WP Cron is spawned.

Improve RSS feed caching

Trac: [#63717](#), [#63719](#)

In WordPress 6.7, an upgrade to the SimplePie library broke transient caching. Repeated calls to the `fetch_feed()` function wouldn't reuse the previous response, even though it is supposed to be cached for 12 hours. This is now fixed in WordPress 6.9.

Impact: The performance impact is clear when benchmarking the server timing for a post that has the Feed [widget](#) (values are median of 100 requests):

Metric	WP 6.8	WP 6.9	Diff (ms)	Diff (%)
Response Time	266.57	34.74	-231.83	-87.0%
wp-before-template	11.95	7.86	-4.09	-34.2%
wp-template	252.55	25.56	-226.99	-89.9%
wp-filter-the_content	224.5	9.35	-215.15	-95.8%
wp-total	264.56	33.4	-231.16	-87.4%

Eliminate Layout Shift in Video Block

Gutenberg: [PR 70293](#)

The Video block has caused a layout shift when any assigned poster image loads and then another layout shift when the video starts playing. This negatively impacts a site's Cumulative Layout Shift (CLS). To fix this, the selected video's width and height are supplied to the `VIDEO` tag's `width` and `height` attributes. In order to ensure the video block's size is responsive, an inline `aspect-ratio` style is also added to the `<video>` tag with a corresponding `height:auto` in the block's CSS. For more background on this change, see [Eliminating Layout Shifts in the Video Block](#) (personal blog).

Impact: Potential improvement of CLS score from poor to good, especially for portrait videos.

Summary of Performance Improvements

As seen from the previous sections, WordPress 6.9 includes a myriad of improvements to optimize the loading experience for site visitors. It optimizes the loading of scripts and styles, introduces the template enhancement output buffer, avoids HTTP requests, and eliminates layout shifts in the Video block. There are numerous other performance improvements under the hood which this [dev note](#) doesn't cover:

- [Optimize database queries and caching](#) (see dev note for [Consistent Cache Keys for Query Groups](#)).
- [Optimize UTF-8 processing](#).
- [A few others](#).

As has been done in performance [dev notes](#) for previous releases, the following section includes benchmarks for web vitals and server timing to see the improvements and regressions to performance in this release compared to the previous release (6.8).

Performance Metrics

Benchmarking is done with the [commands](#) in [GoogleChromeLabs/wpp-research](#) on a MacBook Pro with an M4 Pro chip. The median value for metrics is shown. The tests are done on two fresh installs of WP 6.8 and WP 6.9-RC1. Both sites have the full theme unit test data installed. The "Block category: Widgets" is trashed due to the above [RSS feed caching issue](#), and the "Block category: Embeds" post is trashed to bypass third-party requests. The Twenty Twenty-One theme's "Excerpt settings" are configured in the [Customizer](#) to show "full text" on archive pages, such as the homepage. This matches the default homepage template for Twenty Twenty-Five. Other themes were not configured.

Web Vitals

Benchmarking of web vitals was done with the following command:

```
npm run research -- benchmark-web-vitals --url=http://wp68.local/ --url=http://wp69.local/ --number=100 --network-conditions=broadband --diff
```

"Sample Page"

The average LCP improvement for all classic themes was **-4%**, whereas for block themes the was **-25%**. See [full results](#).

Classic Theme (Twenty Twenty-One):

Metric	WP 6.8	WP 6.9	Diff (ms)	Diff (%)
FCP	186.35	175.3	-11.05	-5.9%
LCP	186.35	175.3	-11.05	-5.9%
TTFB	27.45	29.4	+1.95	+7.1%
LCP-TTFB	158.55	145.65	-12.90	-8.1%

Block Theme (Twenty Twenty-Five):

Metric	WP 6.8	WP 6.9	Diff (ms)	Diff (%)
FCP	141.5	94.7	-46.80	-33.1%
LCP	241.55	195.8	-45.75	-18.9%
TTFB	36.8	38.4	+1.60	+4.3%
LCP-TTFB	203.65	156.9	-46.75	-23.0%

“Block category: Common”

There are 23 unique blocks in the content, which means there is significantly more CSS added compared with the Sample Page. The average LCP improvement for classic themes is reduced to -2.8% and the average reprovement for block themes is -5.8%. Nevertheless, for at least for Twenty Twenty-One and Twenty Twenty-Five, the improvement to LCP is roughly the same as for “Sample Page”. What’s curious is that for the Twenty Twenty-Eleven theme, specifically, the LCP has regressed by +5%. Similarly, the LCP improvement is erased for Twenty Seventeen (a classic theme) and Twenty Twenty-Three (a block theme). Nevertheless, all other themes show LCP improvements. The regression in LCP for the Twenty Eleven theme is worth investigating (even though the theme is relatively unpopular). See [full results](#).

Classic Theme (Twenty Twenty-One):

Metric	WP 6.8	WP 6.9	Diff (ms)	Diff (%)
FCP	237.55	223.4	-14.15	-6.0%
LCP	237.55	223.4	-14.15	-6.0%
TTFB	35	38.7	+3.70	+10.6%
LCP-TTFB	202.65	184.7	-17.95	-8.9%

Block Theme (Twenty Twenty-Five):

Metric	WP 6.8	WP 6.9	Diff (ms)	Diff (%)
FCP	212.1	164.1	-48.00	-22.6%
LCP	305.35	250.35	-55.00	-18.0%

Metric	WP 6.8	WP 6.9	Diff (ms)	Diff (%)
TTFB	47.5	49.3	+1.80	+3.8%
LCP-TTFB	257.4	199.35	-58.05	-22.6%

Server Timing

Benchmarking server timing shows regressions for both `wp-before-template` and `wp-template`. Both have the Performance Lab plugin active with its output buffering enabled (to obtain the `wp-template` timing). The following command was used:

```
npm run research -- benchmark-server-timing --url=http://wp68.local/ --url=http://wp69.local/ --number=1000 --diff --output=md
```

Remember that the server timing is less important than the ultimate LCP metric.

Classic Theme (Twenty Twenty-One)

The “Excerpt Settings” in the Customizer are modified to show “Full text”.

Metric	WP 6.8	WP 6.9	Diff (ms)	Diff (%)
Response Time	24.68	27.63	+2.95	+12.0%
wp-before-template	4.48	5.48	+1.00	+22.3%
wp-template	18.69	20.65	+1.96	+10.5%
wp-total	23.17	26.14	+2.97	+12.8%

With the disabling of loading of separate block styles on demand and yet the template enhancement output buffer forcibly-enabled:

```
add_filter( 'should_load_separate_core_block_assets', '__return_false', 100 );
add_filter( 'wp_should_output_buffer_template_for_enhancement', '__return_true' );
```

Re-running the above benchmarks shows that the source of the regression is largely due to inlining the separate block styles on demand:

Metric	WP 6.8	WP 6.9	Diff (ms)	Diff (%)
Response Time	25.01	26.69	+1.68	+6.7%
wp-before-template	4.5	4.93	+0.43	+9.6%
wp-template	19.01	20.38	+1.37	+7.2%
wp-total	23.51	25.31	+1.80	+7.7%

Re-running the benchmarks without the template enhancement output buffer results in roughly the same numbers. The regression percentages are much closer as the regressions for a block theme (below), which already inlines block styles on demand. This indicates that performance of CSS inlining needs to be optimized, which will benefit block themes and now classic themes as well.

Block Theme (Twenty Twenty-Five)

Metric	WP 6.8	WP 6.9	Diff (ms)	Diff (%)
Response Time	38.16	40.03	+1.87	+4.9%

Metric	WP 6.8	WP 6.9	Diff (ms)	Diff (%)
wp-before-template	8.62	8.97	+0.35	+4.1%
wp-template	27.93	29.31	+1.38	+4.9%
wp-total	36.56	38.35	+1.79	+4.9%

What’s next?

One improvement originally included in the [6.9 performance roadmap](#) was implementing [instant page navigations from browser history via bfcache](#). This was punted from 6.9 due to a Chromium [bug](#), but hopefully it will be fixed so it can be considered for 7.0.

To see what’s coming next for performance, please install the [Performance Lab](#) plugin which will keep you apprised on the latest performance feature plugins that the [Core Performance Team](#). We have biweekly meetings in the [#core-performance Slack](#) channel. Take a look at the [WordPress/performance](#) repo for what is currently being worked on. Now that the template enhancement output buffer has landed in core, we may start working on enhancements that this [capability](#) makes possible.

Props [@jorbin](#) and [@desrosj](#) for review and proofreading.

Tags: [#6-9](#), [#core](#), [#core-performance](#), [#performance](#), [#css](#), [#javascript](#), [#dev-notes](#), [#dev-notes-6-9](#)

Login to Reply

 **Mike - WP Bullet** 1:49 pm on November 18, 2025

Thank you so much for your efforts on this @westonruter!

It’s amazing to see performance being more part of the [core](#) WordPress experience and it being a priority.

Funnily enough I had a client a few years ago who came to me with this exact scam and I [published a case study](#) on it. I didn’t publicly name WP Rocket for fear of backlash but I’m glad you called them out and hold them accountable.

Good to expose these kind of technical tricks that are like smoke and mirrors which give you a false sense of performance comfort but are ultimately just lying and hiding the underlying issues like heavy [JavaScript](#).

Looking forward to what’s next and would love to contribute.

-Mike from WP Bullet

 **user50094473** 10:37 am on December 4, 2025

@westonruter many thanks for your great work. Nonetheless, this wordpress 6.9 update and optimizations broke the style of my website by re-adding the inline global css that I have removed like this:

```
1 function remove_global_styles(){
2     wp_dequeue_style( '&#039;global-styles&#039;; ');
3     wp_dequeue_style( '&#039;wp-block-library&#039;; ');
4     wp_dequeue_style( '&#039;wp-block-library-theme&#039;; ');
5 }
```

To make it working again after this wordpress 6.9 update, I used your code:

```
1 add_filter( '&#039;should_load_separate_core_block_assets&#039;;, '&#039;__return_false&#039;;, 100 );
2 add_filter( '&#039;wp_should_output_buffer_template_for_enhancement&#039;;, '&#039;__return_true&#039;; );
```

What would be the best practice about removing all these inline style in wordpress 6.9 ?

 **Weston Ruter** 4:53 pm on December 4, 2025

@user50094473 Only the `should_load_separate_core_block_assets` [filter](#) should needed to restore the previous behavior. I do not recommend filtering `wp_should_output_buffer_template_for_enhancement` because that will prevent any future performance enhancements from being able to be applied.

 **user50094473** 3:49 pm on December 8, 2025

@westonruter Many thanks for your swift reply. With all these new optimizations released in wp 6.9, would that make sense to continue to use the [plugin Autoptimize](#) to minify css and [javascript](#)?



Weston Ruter 12:07 am on December 9, 2025

@user50094473 The updates in 6.9 do not cancel out the need for plugins like Autoptimize, if you find that your site benefits from the additional optimizations provided. WP 6.9 does not do any runtime minification of JS & CSS, or many of the other performance features provided by optimization plugins.



Louis Wolmarans 10:51 am on December 4, 2025

Hi **@westonruter**, Thank you for the amazing work!

We seem to have an issue with the style hoisting. I tried to best describe my findings below. I might not be 100% accurate in the root cause or suggested fix, but in general, plugins that enqueue CSS during content rendering (common pattern for widgets, shortcodes, or template parts) have their styles marked “done” but never output.

Summary

The new template enhancement output buffer in WordPress 6.9 (`wp_hoist_late_printed_styles()`) does not capture late-enqueued styles that are registered for the head group (0). Only footer group (1) styles are captured via `do_footer_items()`.

Steps to Reproduce

- Enqueue a stylesheet after `wp_head` has run (e.g., during the `_content` rendering)
- The style is enqueued normally via `wp_enqueue_style()` (defaults to group 0)
- Style appears in `$wp_styles->done` but is missing from HTML output

Expected Behavior

Late-enqueued styles should be hoisted to regardless of their group.

Actual Behavior

Only group 1 (footer) styles are captured:

```
// wp-includes/script-loader.php line 3702-3704
ob_start();
wp_styles()->do_footer_items(); // Only processes group 1!
$printed_late_styles = ob_get_clean();
```

Impact

Plugins that enqueue CSS during content rendering (common pattern for widgets, shortcodes, or template parts) have their styles marked “done” but never output.

Suggested Fix

Modify `wp_hoist_late_printed_styles()` to also capture late-enqueued head-group styles, or provide a mechanism for plugins to register styles for hoisting.

Workaround

Plugins must implement their own fallback using `wp_template_enhancement_output_buffer` filter to detect and insert missing styles.

Reference: <https://github.com/elementor/elementor/pull/33721/files>



Weston Ruter 4:50 pm on December 4, 2025

@louiswol94 Thanks for the report. We’ll need to get this filed in Trac, but in the meantime, I’m trying to reproduce the issue. I created a test plugin: <https://gist.github.com/westonruter/3772371f6c6aa9fff59f038aee58e87c>

The plugin registers and enqueues a stylesheet at `wp_body_open` (and so after `wp_head` has run). The stylesheet is printed for me in both a block theme and classic theme. So I cannot reproduce the issue you’re reporting. Please fork my Gist and modify it so that the issue can be reproduced in a core theme, namely Twenty Twenty-One (a classic theme) and/or Twenty Twenty-Five (a block theme).



weavingw 2:49 am on December 5, 2025

The 6.9 update completely broke all the styles on a clients site. I don’t understand how you would implement these kinds of changes in the `wp-includes` and not check if the default wordpress templates are being used or not.

obviously, if we’re using a custom coded theme, last thing you would want it to make overrides to the site..?

`global-styles-inline-css` one of the culprits and then you have `wp-include` styles being pulled in breaking the site.



Weston Ruter 3:06 am on December 5, 2025

@weavingw I'm not sure exactly what you mean, but naturally we did test the changes with the default themes. The changes were part of all beta releases and release candidates for 6.9, and any reported issues were fixed during testing. If you can file a [ticket](#) with details on the breakages you're seeing and how to reproduce the issue, this will be helpful.



Rick Curran 1:01 pm on December 5, 2025

@westonruter I added

```
1 | add_filter( 'should_load_separate_core_block_assets', '__return_false', 100 );
```

And I can confirm that this resolved the issues I was seeing on one of my sites that was having layout issues since 6.9.